

SWIFT

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ *Red star = Important Topic (read carefully, exam / interview favorite)*

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to Swift

★ 2. Variables - let vs var

- 3. Data Types

★ 4. Optionals

- 5. Conditional Statements (if, else if, else)
- 6. Loops - for, while

★ 7. Functions

- 8. Closures

★ 9. Classes and Structs

- 10. Inheritance
- 11. Protocols

★ 12. Arrays and Dictionaries

- 13. Error Handling (do, try, catch)
- 14. Enums

★ 1. Introduction to Swift *[IMPORTANT TOPIC]*

Explanation:

Swift is a modern programming language made by Apple, used to build apps for iPhone, iPad, and Mac. It is designed to be safe, fast, and easier to read than Apple's older language, Objective-C.

Key Points:

- Official language for iOS and macOS app development.
- Designed to prevent common programming mistakes automatically.
- Uses `print()` to display output.

Example Code:

```
print("Hello, World!")
```

Output:

```
Hello, World!
```

★ 2. Variables - let vs var [IMPORTANT TOPIC]

Explanation:

Swift has two ways to create a variable. 'let' creates a value that CANNOT be changed once set (a constant). 'var' creates a variable whose value CAN be changed later.

Example Code:

```
let name = "Pravendra" // cannot be changed
var age = 21           // can be changed

age = 22
print("\(name) is \(age) years old")
```

Output:

```
Pravendra is 22 years old
```

💡 Note: `\( )` inside a string is used to insert a variable's value — this is called 'string interpolation'.

3. Data Types

Explanation:

Swift has clear, simple data types, and it can usually detect a variable's type automatically without us writing it directly.

Key Points:

- Int -> whole numbers
- Double -> decimal numbers
- String -> text
- Bool -> true or false

Example Code:

```
let age: Int = 21
let height: Double = 5.9
let isStudent: Bool = true
print(age)
```

Output:
21

★ 4. Optionals [IMPORTANT TOPIC]

Explanation:

An Optional is Swift's way of handling a value that MIGHT be missing (nil). A normal variable in Swift can never be nil (empty) unless we specifically mark it as an Optional using a ? symbol. This prevents many common app crashes.

Example Code:

```
var nickname: String? = nil    // Optional - may or may not have a value
print(nickname)

nickname = "Pravi"
if let realName = nickname {
    print("Nickname is \(realName)")
}
```

Output:

```
nil
Nickname is Pravi
```

💡 Note: 'if let' safely 'unwraps' an Optional — the code inside only runs if the value actually exists.

5. Conditional Statements (if, else if, else)

Explanation:

Conditional statements let the program make decisions based on whether something is true or false.

Example Code:

```
let marks = 75

if marks >= 90 {
    print("Grade A")
} else if marks >= 60 {
    print("Grade B")
} else {
    print("Grade C")
}
```

Output:

Grade B

6. Loops - for, while

Explanation:

Loops repeat a block of code multiple times. Swift's 'for' loop commonly uses ranges, like 1...3, to repeat a fixed number of times.

Example Code:

```
for i in 1...3 {  
    print("Count: \(i)")  
}
```

Output:

```
Count: 1  
Count: 2  
Count: 3
```


★ 7. Functions *[IMPORTANT TOPIC]*

Explanation:

A function is a reusable block of code, defined using the 'func' keyword. Swift functions can label their parameters, making function calls very readable.

Example Code:

```
func addNumbers(a: Int, b: Int) -> Int {  
    return a + b  
}  
  
let total = addNumbers(a: 5, b: 7)  
print("Total: \(total)")
```

Output:

Total: 12

8. Closures

Explanation:

A closure is a self-contained block of code that can be passed around and used elsewhere, similar to a lambda function in other languages. Closures are widely used for tasks like handling button taps or completing a background task.

Example Code:

```
let square = { (n: Int) -> Int in
    return n * n
}

print(square(5))
```

Output:
25

★ 9. Classes and Structs [IMPORTANT TOPIC]

Explanation:

Both classes and structs let us group related data and functions together. The key difference is: a class is a 'reference type' (shared, changes affect all copies), while a struct is a 'value type' (copied fully each time). Swift encourages using structs where possible, for better safety.

Example Code:

```
struct Car {  
    var brand: String  
    func showBrand() {  
        print("Brand: \(brand)")  
    }  
}  
  
let c1 = Car(brand: "Toyota")  
c1.showBrand()
```

Output:

Brand: Toyota

10. Inheritance

Explanation:

Inheritance lets one CLASS reuse the properties and functions of another class. Note: structs in Swift CANNOT use inheritance — only classes can.

Example Code:

```
class Animal {  
    func eat() { print("This animal eats food") }  
}  
  
class Dog: Animal {  
    func bark() { print("The dog barks") }  
}  
  
let d = Dog()  
d.eat()  
d.bark()
```

Output:

```
This animal eats food  
The dog barks
```

11. Protocols

Explanation:

A protocol defines a set of rules (methods and properties) that a class or struct **MUST** follow if it agrees to use that protocol. It is similar to an 'interface' in other languages, and is used heavily throughout Swift and iOS development.

Example Code:

```
protocol Greetable {  
    func greet()  
}  
  
struct Person: Greetable {  
    func greet() {  
        print("Hello!")  
    }  
}  
  
let p = Person()  
p.greet()
```

Output:
Hello!

★ 12. Arrays and Dictionaries [IMPORTANT TOPIC]

Explanation:

An Array stores an ordered list of values. A Dictionary stores data as key-value pairs, similar to a real dictionary.

Example Code:

```
var fruits = ["apple", "banana", "mango"]
fruits.append("orange")
print(fruits[0])

var marks = ["Pravendra": 90, "Aman": 85]
print(marks["Pravendra"]!)
```

Output:

```
apple
90
```

💡 *Note: The ! symbol force-unwraps an Optional — use it only when you are SURE the value exists, or the app may crash.*

13. Error Handling (do, try, catch)

Explanation:

Swift uses do-try-catch blocks to safely handle errors that a function might 'throw' while running, instead of the app crashing unexpectedly.

Example Code:

```
enum MyError: Error {
    case divideByZero
}

func divide(_ a: Int, _ b: Int) throws -> Int {
    if b == 0 { throw MyError.divideByZero }
    return a / b
}

do {
    let result = try divide(10, 0)
    print(result)
} catch {
    print("Error: Cannot divide by zero!")
}
```

Output:

Error: Cannot divide by zero!

14. Enums

Explanation:

An enum (enumeration) lets us define a fixed set of related, named values, making code more readable and safer than using plain numbers or text.

Example Code:

```
enum Direction {  
    case north, south, east, west  
}  
  
let myDirection = Direction.north  
  
switch myDirection {  
case .north:  
    print("Heading North")  
default:  
    print("Some other direction")  
}
```

Output:

Heading North